



Gen-Knob Tuner for Relational Databases

Progress Report

Authors:

Madara Mendis

Madushan Suriyanbandara

Nisith Dissanayake

Nithika Dias

External Supervisor:

Dr. Srinath Perera

Internal Supervisor:

Dr. Buddhika Karunaratne

University of Moratuwa

March 31, 2026

Abstract

Relational Database Management Systems (RDBMS) rely on hundreds of configuration parameters, or "knobs," that significantly impact performance. Finding optimal settings for specific workloads traditionally requires exhaustive manual tuning or resource-heavy automated tools. This project explores the development of Gen-Knob Tuner, an end-to-end database tuning framework based on the E2ETune architecture. Unlike existing solutions that require substantial GPU resources, our research focuses on achieving competitive performance improvements, targeting the ~51% gain reported in literature, using lightweight Large Language Models (LLMs) such as Qwen2.5 (7B). We demonstrate a methodology for workload generation, label collection via Gaussian Process models, and the training of a performance cost model. Preliminary results show our cost model achieves an R^2 score of 0.82, paving the way for efficient, CPU-only knob tuning in resource-constrained environments.

Contents

1	Introduction	5
1.1	Background	5
1.2	Problem Statement	5
1.3	Research Objectives	6
2	Literature Review	6
2.1	Evolution of Database Tuning	6
2.2	Traditional and Heuristic-Based Tuning	7
2.3	Bayesian Optimization (BO) Approaches	8
2.3.1	HEBO: Heteroscedastic and Evolutionary Bayesian Optimization	8
2.4	Reinforcement Learning (RL) Approaches	9
2.5	Deep Learning (DL)-Based Knob Tuning	9
2.6	Large Language Models (LLMs) in Database Tuning	10
2.6.1	DB-BERT and GPTuner: LM-Assisted Iterative Tuning	10
2.6.2	E2ETune: End-to-End Knob Tuning	10
3	Methodology and Experimental Evaluation	11
3.1	System Architecture	11
3.2	Data Collection and Labeling	11
3.2.1	Benchmark Selection	12
3.2.2	Data Collection Strategy	12
3.2.3	Training the Cost Model	13
3.2.4	Mass Labeling via Surrogate Optimization	15
3.3	Feature Engineering	15
3.3.1	Database Configuration Knobs (10-15)	16
3.3.2	Internal Metrics (15)	17
3.3.3	Workload and Query Plan Features (40)	18
3.4	SLM Fine-tuning and Knowledge Distillation	19
3.4.1	Model Selection: Qwen 2.5-Coder-7B	19
3.4.2	Supervised Fine-Tuning (SFT) Strategy	20
3.5	RAG-Enhanced Recommendations	21
3.5.1	Knowledge Base Construction	21
3.5.2	RAG Integration and Explainability	21
4	Implementation Details	22
4.1	Core Database Interaction: <code>Database.py</code>	22
4.1.1	Connection Management	22

4.1.2	Knob Application Logic	22
4.2	The Tuning Loop: <code>stress_testing_tool.py</code>	23
4.2.1	Multi-threaded Workload Execution	23
4.2.2	Surrogate Integration	23
4.3	Feature Extraction: <code>get_query_plans.py</code>	23
4.4	Surrogate Model Implementation: <code>surrogate.py</code>	24
4.5	Configuration Management: <code>config.ini</code>	24
4.6	SLM Inference with <code>Llama.cpp</code>	24
5	Evaluation and Results	25
5.1	Inference Efficiency	25
5.2	Optimization Effectiveness	25
5.3	Analysis of the Performance Gap	26
6	Discussion and Challenges	28
6.1	In-depth Analysis of Preliminary Results	28
6.2	Challenges Encountered and Mitigation Strategies	28
6.2.1	Representative Subset Selection for Large Benchmarks	28
6.2.2	Knob Space Complexity and Optimizer Convergence	28
6.2.3	Resource Constraints and Model Quantization Effects	29
7	Future Work	29
7.1	Short-Term Goals (Next 3 Months)	29
7.1.1	Completion of SLM Fine-tuning	29
7.1.2	Full Integration of the RAG System	29
7.1.3	Expanded Workload Coverage	29
7.2	Long-Term Vision and Research Directions	30
7.2.1	Cross-Database Engine Adaptation	30
7.2.2	Real-time Dynamic and Adaptive Tuning	30
8	Conclusion	30
8.1	Ethical Considerations and Societal Impact	31
8.1.1	Bias and Fairness	31
8.1.2	Job Displacement and Augmentation	31
8.1.3	Security and Privacy Implications	32
8.1.4	Environmental Impact	32

List of Figures

1	Data collection pipeline: We select 13 representative workloads per benchmark and run HEBO optimization on the actual database to generate ground-truth performance data.	13
2	Cost Model Architecture: A Voting Regressor combining Random Forest and Gradient Boosting predicts performance based on query plans and knob configurations.	14
3	HEBO optimization is done using the cost model instead of running every iteration on the database.	15
4	Comparison of Average CPU Inference Time: Our model achieves a 5x speedup over the SOTA baseline.	26
5	Performance Improvement by Benchmark: Our model outperforms E2ETUNE on SSB and remains competitive on average.	27

List of Tables

1	Cost Model Evaluation on Test Split	15
2	Primary Configuration Knobs Selected for Tuning	17
3	Comparison of Candidate Base Models	19
4	Loss Comparison: Direct Config vs. Binned Output	21
5	Performance Improvement Comparison (Percentage over Default)	26
6	Experimental Constraints and Data Differences	27

1 Introduction

1.1 Background

Relational Database Management Systems (RDBMS) are the backbone of modern data-driven applications, handling everything from simple transactional records to complex analytical processing. To maintain performance across diverse environments, almost all RDBMS's expose a large range of configuration parameters, commonly known as knobs. These knobs control critical system behaviors, including memory allocation (e.g., `shared_buffers`), background writer frequency, and query planner constants. The performance of a DBMS is highly sensitive to these configurations. A well-tuned database can process queries orders of magnitude faster than one with default settings. However, the optimal configuration is not static; it depends heavily on the underlying hardware, the database schema, and the specific characteristics of the workload [1].

Historically, database administrators (DBAs) relied on commonly known rules or manual trial-and-error to tune these settings. However, as data volumes grow and workloads become more dynamic, the high-dimensional search space of these parameters makes manual optimization nearly impossible for human experts to manage effectively.

1.2 Problem Statement

Despite the importance of knob tuning, it remains a significant challenge for several reasons:

- **High Dimensionality:** Modern DBMS have hundreds of tunable knobs, creating an astronomical search space. The number of possible configurations can easily exceed 10^{100} , making exhaustive search infeasible [2].
- **Non-linear Interactions:** Knobs often interact in complex, non-linear ways. For instance, increasing memory for one component might starve another, leading to degraded performance. Understanding these intricate relationships requires deep domain expertise and extensive experimentation.
- **Workload Dependency:** A configuration that works well for an Online Analytical Processing (OLAP) workload might be disastrous for an Online Transaction Processing (OLTP) workload. The optimal settings are highly dependent on the specific query patterns, data access methods, and concurrency levels of the active workload.
- **Resource Intensity:** Existing automated tuning methods, such as Bayesian Optimization (BO) and Reinforcement Learning (RL), typically require numerous "workload replays" to evaluate different configurations. Each replay can take minutes or even hours, making the tuning process extremely slow and resource-intensive [3].

- **Hardware Requirements:** Recent state-of-the-art methods like E2ETune [1] utilize Large Language Models (LLMs) that require expensive GPU resources for inference, which may not be available in all production or edge environments. This limits their practical applicability for many organizations.

1.3 Research Objectives

The Gen-Knob Tuner project, also known as **AI4DB**, aims to bridge the gap between high-performance AI-based tuning and practical, resource-efficient deployment. Our primary objectives are:

- Replicate the performance gains of E2ETune ($\sim 51\%$ improvement) using a much lighter model architecture.
- Enable CPU-only inference by utilizing Small Language Models (SLMs), making AI-native tuning accessible without specialized hardware. This is crucial for deployment in resource-constrained environments.
- Develop a high-fidelity surrogate cost model to predict database performance, thereby reducing the need for expensive workload replays. The goal is to achieve an R^2 score of over 0.80, accurately reflecting real-world performance.
- Leverage Retrieval-Augmented Generation (RAG) to incorporate expert knowledge and documentation into the tuning process. This enhances the generalizability and reliability of the generated recommendations.

2 Literature Review

Our research is primarily grounded in two foundational areas: end-to-end generative tuning and the challenges of cloud-native database configuration. This section provides a comprehensive overview of the state-of-the-art in database tuning, from traditional methods to modern AI-driven approaches.

2.1 Evolution of Database Tuning

The field of database tuning has evolved significantly over the decades, driven by increasing data volumes, complex workloads, and the advent of machine learning. Initially, tuning was a manual, art-like process heavily reliant on the experience and intuition of Database Administrators (DBAs). With the growth of relational databases, rule-based systems and cost-based optimizers emerged, providing some automation. The current frontier involves sophisticated machine learning and generative AI techniques to automate and optimize database performance [1].

2.2 Traditional and Heuristic-Based Tuning

Heuristic-based methods explore the configuration search space using manually crafted rules or predefined heuristics, typically derived from DBAs’ experience and vendor documentation. While these approaches can be practical and fast to apply, they usually require substantial human intervention and tend to exhibit limited search efficiency, which can lead to suboptimal performance.

Heuristic-based tuning methods can be broadly categorized into **rule-based** and **search-based** methods.

- **Rule-based methods:** These methods codify expert knowledge into a set of tuning rules (or “what-if” questions) that guide the selection of knob values. For example, PostgreSQL documentation suggests that a reasonable starting point for `shared_buffers` is approximately 25% of system memory, while MySQL guidance often places `innodb_buffer_pool_size` between 50% and 75% of system memory. Tools such as PG Tune use basic user-provided inputs (e.g., database type and hardware specification) to recommend settings. In practice, the database version and CPU count can directly affect which knobs are available and how they should be set (e.g., the availability of parallelism-related knobs differs across PostgreSQL versions, and worker-related knob values are commonly tied to the number of CPU cores). Probabilistic reasoning has also been proposed to determine whether a given rule is applicable under a particular workload and system state, and to handle correlations between multiple rules by explicitly modeling uncertainty.
- **Search-based methods:** These methods apply heuristic search procedures to locate a near-optimal configuration. A typical workflow partitions the knob space into subspaces, evaluates candidate subspaces by running the workload, selects the most promising region, and then refines the solution by searching neighboring configurations. Representative approaches include ensemble strategies that combine multiple search techniques (e.g., multi-armed bandits and particle swarm optimization) and sampling-optimization methods that first sample configurations and then apply local improvement around strong candidates. Although these methods can adapt across systems and often find configurations close to the best observed settings, they are time-consuming due to repeated sampling and evaluation, and they may miss better configurations if parts of the search space are not explored.

Overall, heuristic methods provide a useful baseline and can outperform purely manual tuning in terms of speed. However, due to complex knob interactions and workload drift, they often struggle to identify globally optimal settings and to remain robust under changing workloads and data distributions.

2.3 Bayesian Optimization (BO) Approaches

Bayesian Optimization (BO) is widely used for black-box knob tuning, where evaluating a configuration requires executing a workload and measuring performance. BO-based methods (e.g., VBO, HEBO, and SMAC) learn a *surrogate model* that predicts performance metrics from a set of input features and then use an acquisition function to select the next configuration to evaluate. Different BO frameworks adopt different surrogate models; for example, VBO commonly uses a vanilla Gaussian Process (GP), whereas SMAC is based on Random Forests. In each iteration, the selected configuration is applied to the database, the workload is executed to obtain performance observations, and the surrogate is refined. Over time, the surrogate model becomes more accurate and guides the search toward improved configurations.

Early BO-based knob tuning systems, such as iTuned, relied primarily on GP surrogates. Subsequent work improved surrogate fidelity by enriching the feature set used for prediction. For instance, OnlineTune incorporates workload-side signals such as query arrival rates, query types, and index information. Other systems emphasize system-level attributes to improve robustness across deployments (e.g., memory-control related features across different workloads, container settings, or JVM configurations), while approaches such as ResTune explicitly incorporate resource utilization metrics including CPU, memory, and I/O usage. Despite these advances, a common limitation of BO-based tuning is sample complexity: many methods still require hundreds of iterations to reach a suitable configuration for a new workload, making the tuning process time-consuming and resource-intensive in practice.

2.3.1 HEBO: Heteroscedastic and Evolutionary Bayesian Optimization

HEBO [4] represents the state-of-the-art in sample-efficient hyperparameter optimization and is a critical component of our data collection strategy. It addresses several limitations of traditional BO, making it more robust for complex, real-world tuning problems:

- **Non-stationarity:** It uses input and output warping techniques to handle non-stationary objective functions, where the relationship between knobs and performance might change across different regions of the search space.
- **Heteroscedasticity:** HEBO models noise that varies across the input space, meaning it can account for situations where some configurations produce more stable performance results than others.
- **Multi-objective Ensembles:** It employs an ensemble of acquisition functions to balance exploration and exploitation more effectively, leading to faster convergence to optimal configurations.

In our project, we transitioned from the SMAC optimizer to HEBO due to its superior performance in high-dimensional, noisy search spaces. This allowed us to efficiently collect high-quality training data, even though we limited the knob space to 15 to maintain efficiency and convergence within a reasonable number of iterations.

2.4 Reinforcement Learning (RL) Approaches

RL-based knob tuning methods use reinforcement learning algorithms to iteratively adjust database knobs based on observed system behavior. A common formulation models tuning as a sequential decision-making process, where an agent selects knob configurations (actions) given the current database and workload condition (state), and receives a reward derived from a performance metric (e.g., latency or throughput).

Many systems adopt Deep Deterministic Policy Gradient (DDPG), an actor-critic algorithm designed for continuous action spaces. In this setting, the *actor* proposes a knob configuration for the current state, while the *critic* evaluates the quality of that action and guides policy improvement by optimizing the value function and policy iteratively. CDBTune [3] applies DDPG using runtime metrics as the state representation to generate database configurations. QTune [5] extends this design with a Double-State DDPG architecture that augments the state with query-related features (e.g., query types and the tables involved), enabling more workload-aware tuning decisions.

Despite their ability to adapt to dynamic workloads, RL-based approaches often require many online interactions (iterations) before the policy stabilizes, which can be resource-intensive and may exhibit poor initial performance during the exploration phase.

2.5 Deep Learning (DL)-Based Knob Tuning

Deep learning (DL)-based knob tuning methods train neural networks as learned cost models to predict database performance from input features, enabling faster exploration of candidate configurations by reducing reliance on expensive workload replays. For example, prior work has replaced the Gaussian Process surrogate used in earlier BO systems with deep neural networks to better scale with higher-dimensional knob spaces. Similarly, iBTune trains a deep neural network to estimate response time using system and workload metrics such as logical reads and CPU usage.

In practice, these learned cost models are often only one component of a complete tuning pipeline. While they can accelerate the evaluation of candidate configurations, many systems still require iterative trials (and periodic model updates) to correct prediction error and to remain robust under workload and data distribution shifts.

2.6 Large Language Models (LLMs) in Database Tuning

The most recent and rapidly evolving trend is the application of language models (LMs) to database optimization. Recent work has demonstrated the utility of LMs across a wide range of database tasks, including database diagnosis (e.g., root cause analysis), text-to-SQL, data preparation (e.g., value filling and entity resolution), data integration, query processing, and table question answering. Within knob tuning, LMs are particularly attractive because they can ingest unstructured expert knowledge (manuals, blogs, and forum discussions) and translate it into actionable guidance.

However, many classical tuning methods remain inherently iterative. Bayesian Optimization and Reinforcement Learning approaches typically require repeated online interactions (i.e., applying a configuration, replaying a workload, and updating the model/policy) before converging to a high-quality configuration, which makes them slow and resource-intensive for new workloads.

2.6.1 DB-BERT and GPTuner: LM-Assisted Iterative Tuning

DB-BERT [6] and GPTuner [7] leverage LMs primarily as a knowledge interface—extracting hints and best practices from manuals and web forums to constrain or guide the search space. Nevertheless, these systems still rely on a base tuner (e.g., Bayesian Optimization or Reinforcement Learning) to perform iterative exploration and to produce the final knob configuration.

2.6.2 E2ETune: End-to-End Knob Tuning

While LM-assisted systems improve the quality of iterative search, they still inherit the latency and resource costs of online exploration. E2ETune [1] introduces an end-to-end formulation that trains a language model offline on extensive historical tuning tasks and then directly predicts suitable knob configurations for a given workload. By removing the need for online search during inference, end-to-end LM tuning can produce optimized configurations significantly faster than prior BO/RL-based pipelines, while also achieving strong performance improvements.

Key contributions of E2ETune include:

- **Direct configuration prediction:** A sequence-to-sequence model maps workload features to knob settings without iterative workload replays at inference time.
- **Improved tuning efficiency:** Offline training amortizes the cost of exploration, enabling rapid application to new workloads and different database instances.
- **Practical data generation:** A framework to construct training data by pairing workloads with high-quality configurations for fine-tuning.

Given the above advantages, we adopt the end-to-end LM tuning paradigm as the baseline for our work and focus on reproducing its benefits under resource constraints by fine-tuning a lightweight Small Language Model (SLM).

3 Methodology and Experimental Evaluation

Our methodology is structured around a three-stage pipeline, inspired by the E2ETune framework, but adapted to prioritize resource efficiency and accessibility. This section details each stage, from data construction to model training and end-to-end tuning.

3.1 System Architecture

The AI4DB framework is designed as a modular, end-to-end pipeline for database optimization. The architecture consists of four primary conceptual components: the Data Collection Engine, the Surrogate Cost Model, the Knowledge-Augmented RAG System, and the Fine-tuned Small Language Model (SLM). These components work in concert to achieve efficient and accurate database knob tuning.

1. **Stage I: Data Construction:** This initial stage focuses on setting up different bench marks on Postgresql and collecting performance labels using the HEBO optimizer. A surrogate cost model is trained to predict performance, reducing the need for extensive physical workload execution. This process creates the foundational dataset for training our models.
2. **Stage II: Model Training:** In this stage, we fine-tune a lightweight LLM (Qwen 2.5-Coder-7B) on normalized best configurations derived from Stage I.
3. **Stage III: Generalization:** Implemented a Retrieval-Augmented Generation (RAG) framework backed by a comprehensive technical knowledge base, enabling the system to dynamically adapt and generalize optimization strategies across diverse database versions and configurations without retraining.

3.2 Data Collection and Labeling

The foundation of our approach is a high-quality dataset of workload-configuration pairs, where each pair consists of a specific workload and its corresponding optimal or near-optimal database configuration. This dataset is crucial for training the SLM and the surrogate cost model.

3.2.1 Benchmark Selection

We utilize a diverse set of industry-standard benchmarks to ensure the robustness and generalizability of our models. These benchmarks represent different types of database workloads:

- **JOB (Join Order Benchmark):** Based on the IMDb dataset, JOB focuses on complex join queries, making it ideal for evaluating query optimizer performance and the impact of knobs on join operations. It contains 113 queries across 33 query templates [1].
- **SSB (Star Schema Benchmark):** A classic OLAP benchmark focusing on large-scale data aggregation and analytical queries. It features a star schema data model and 13 queries organized into 4 flights (query groups) [1].
- **TPC-H:** A standard industry benchmark for decision support systems, involving complex ad-hoc queries and concurrent data modifications. It consists of 22 complex SQL queries [1].
- **TPC-DS:** A more complex version of TPC-H, modeling multi-channel retail data and customer demographics. It comprises 99 complex SQL queries across multiple sales channels [1].

3.2.2 Data Collection Strategy

Conducting online optimization (running workloads directly on the database) for every benchmark query is computationally prohibitive and time-consuming. To overcome this bottleneck, we adopted a surrogate modeling approach. Instead of optimizing all available workloads directly, we employed a sampling strategy to create a representative training dataset for a performance prediction model (Cost Model).

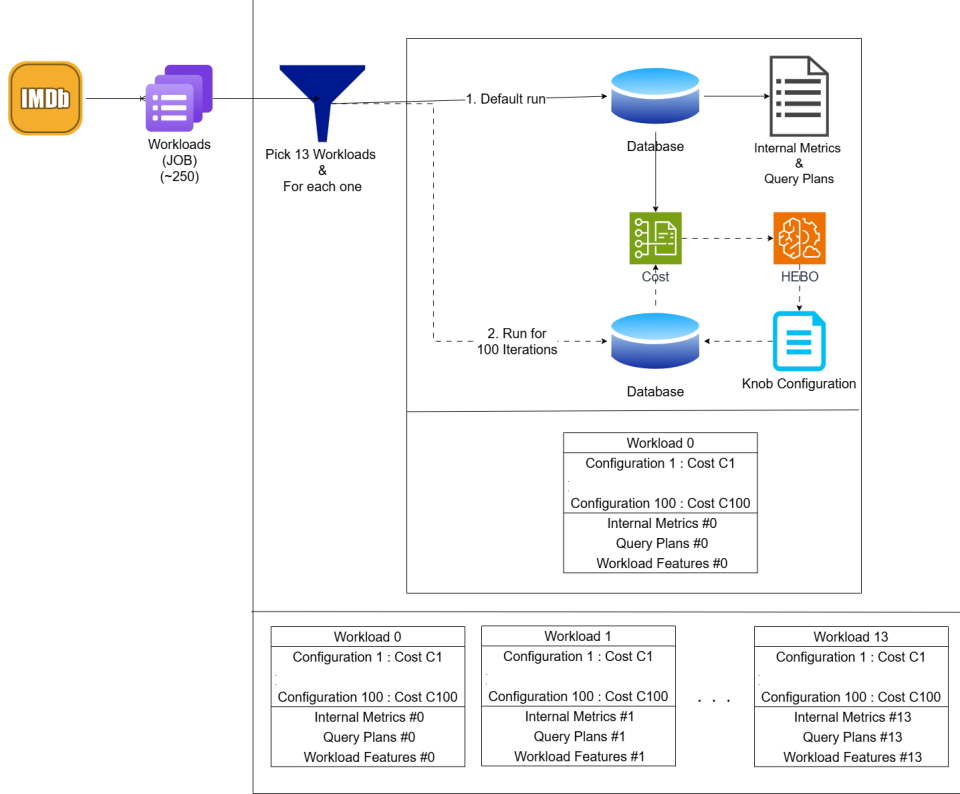


Figure 1: Data collection pipeline: We select 13 representative workloads per benchmark and run HEBO optimization on the actual database to generate ground-truth performance data.

The process involves the following steps:

- 1. Workload Selection:** We select a subset of 13 representative workloads from each benchmark (e.g., Join Order Benchmark (JOB), IMDb).
- 2. Ground Truth Generation:** For each selected workload, we execute the iterative HEBO optimization loop on a live PostgreSQL instance for 100 iterations. This process captures the mapping between specific knob configurations (C_i) and the resulting performance metrics (e.g., Queries Per Second).
- 3. Dataset Aggregation:** The result is a high-fidelity dataset containing approximately 1,300 data points per benchmark (13 workloads $\times$ 100 iterations), totaling 5,200 data points across all benchmarks. This dataset serves as the ground truth for training our Cost Model.

3.2.3 Training the Cost Model

To generalize performance prediction to unseen workloads without requiring database interaction, we trained a dedicated Cost Model. This model acts as a surrogate for the database, allowing us to estimate the performance of a configuration in milliseconds.

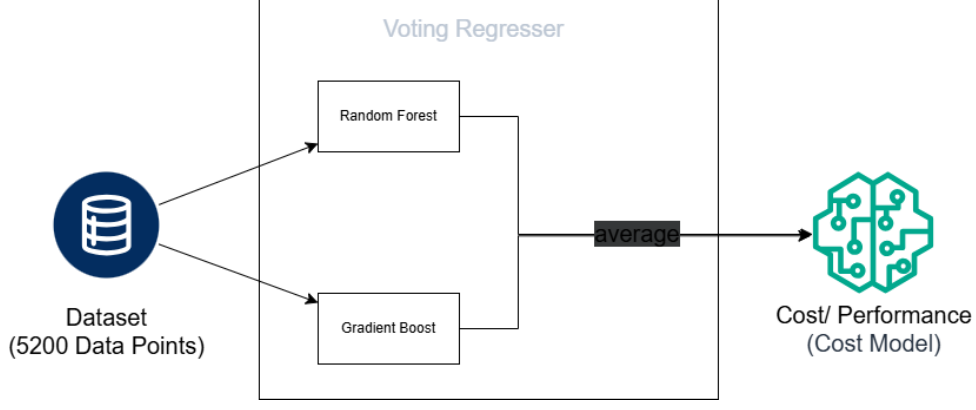


Figure 2: Cost Model Architecture: A Voting Regressor combining Random Forest and Gradient Boosting predicts performance based on query plans and knob configurations.

Model Architecture: As illustrated in Figure 3, the Cost Model is implemented as a **Voting Regressor** ensemble, which averages the predictions of two distinct estimators to improve robustness and reduce overfitting:

- **Random Forest Regressor:** Captures non-linear dependencies between database knobs.
- **Gradient Boosting Regressor:** Optimizes for residual errors in prediction.

Input Features: The model is trained on a comprehensive feature set designed to capture the database state:

- **Knob Configurations:** The numerical values of the tunable parameters.
- **Query Plans:** Vectorized representations of the execution plans (e.g., Seq Scan, Index Scan).
- **Workload Features:** Statistical properties of the query (e.g., I/O usage, read/write ratio).

Target Variable: The model predicts **Queries Per Second (QPS)** (normalized) as the primary performance metric.

Performance Evaluation: We evaluated the Cost Model on a hold-out test set (1,040 samples). As shown in Table 1, our approach achieves an R^2 score of **0.8267** and a Spearman’s Rank Coefficient of **0.8092**, demonstrating strong correlation with actual database performance and effectively replacing the need for online execution during the labeling phase.

Table 1: Cost Model Evaluation on Test Split

Metric	R2 Score	MAE	RMSE	Spearman’s Coeff.
Ours	0.8267	0.1849	0.2218	0.8092

3.2.4 Mass Labeling via Surrogate Optimization

With the trained Cost Model, we can now efficiently label the remaining majority of workloads that were not run on the database.

Instead of executing SQL queries on PostgreSQL, we utilize the Cost Model as the objective function for the HEBO optimizer. For every remaining workload in our corpus, HEBO queries the Cost Model to find the optimal configuration. This allows us to generate ”best configuration” labels for thousands of queries in a fraction of the time, creating the large-scale dataset required for fine-tuning the Large Language Model (Qwen 2.5).

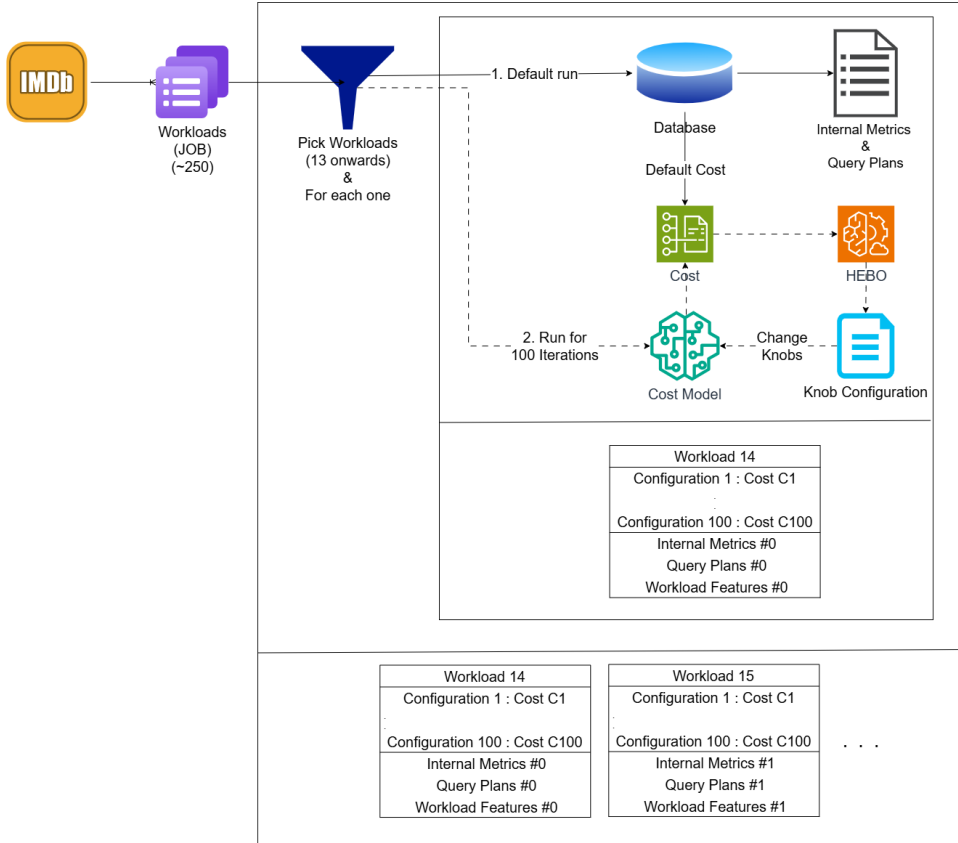


Figure 3: HEBO optimization is done using the cost model instead of running every iteration on the database.

3.3 Feature Engineering

A critical contribution of our work is the refinement of the feature set used for performance prediction in the surrogate cost model and as input to the SLM. We have identified

65 key features categorized into four groups, providing a comprehensive representation of the database state and workload characteristics.

3.3.1 Database Configuration Knobs (10-15)

We focus on a carefully selected subset of 10-15 highly impactful PostgreSQL knobs. This reduction from E2ETune's 45 knobs was necessary to ensure the HEBO optimizer could converge efficiently within a reasonable number of iterations and to manage the complexity for our lightweight SLM. These knobs are chosen based on their known significant impact on performance across various workloads.

Table 2: Primary Configuration Knobs Selected for Tuning

Knob Name	Description	Type
<code>shared_buffers</code>	Sets the amount of memory dedicated to database data caching. A larger value can reduce disk I/O.	Integer
<code>work_mem</code>	Specifies the amount of memory to be used by internal sort operations and hash tables before writing to temporary disk files.	Integer
<code>maintenance_work_mem</code>	Maximum amount of memory to be used by maintenance operations, such as VACUUM, CREATE INDEX, and ALTER TABLE ADD FOREIGN KEY.	Integer
<code>effective_cache_size</code>	PostgreSQL's estimate of the total amount of disk cache available to the operating system and within the database itself. Influences query planner decisions.	Integer
<code>random_page_cost</code>	Sets the planner's estimate of the cost of a non-sequentially fetched disk page. Lower values encourage index scans.	Float
<code>cpu_tuple_cost</code>	Sets the planner's estimate of the cost of processing each tuple during a query.	Float
<code>cpu_index_tuple_cost</code>	Sets the planner's estimate of the cost of processing each index entry during an index scan.	Float
<code>seq_page_cost</code>	Sets the planner's estimate of the cost of a sequentially fetched disk page.	Float
<code>effective_io_concurrency</code>	Sets the number of concurrent disk I/O operations that PostgreSQL expects to be able to execute simultaneously.	Integer
<code>max_worker_processes</code>	Maximum number of background processes that the system can support.	Integer

3.3.2 Internal Metrics (15)

These features capture the runtime state and behavior of the database during workload execution. They provide crucial insights into how the current configuration is interacting with the workload. Examples include:

- `blks_hit`: Number of block hits in the buffer cache, indicating cache efficiency.
- `blks_read`: Number of blocks read from disk, indicating disk I/O activity.

- `tup_returned`: Number of tuples (rows) returned by queries.
- `tup_fetched`: Number of tuples fetched by queries.
- `xact_commit`: Number of transactions committed.
- `xact_rollback`: Number of transactions rolled back.
- `deadlocks`: Number of deadlocks detected.
- `temp_bytes`: Total amount of data written to temporary files by queries.
- `temp_files`: Number of temporary files created.
- `buffers_alloc`: Number of buffers allocated.
- `buffers_backend`: Number of buffers written directly by a backend.
- `buffers_clean`: Number of buffers written by the background writer.
- `buffers_checkpoint`: Number of buffers written during checkpoints.
- `checkpoints_req`: Number of requested checkpoints.

These metrics are collected using PostgreSQL's built-in statistics views (e.g., `pg_stat_database`) and provide a real-time snapshot of system health and performance bottlenecks.

3.3.3 Workload and Query Plan Features (40)

We extract structural features from the PostgreSQL `EXPLAIN (FORMAT JSON)` plans. This is a sophisticated process that transforms the hierarchical query plan into a fixed-length numerical vector. This includes:

- **Plan Parsing:** The JSON output of PostgreSQL's `EXPLAIN` is parsed to extract detailed information about each operation.
- **Recursive Traversal:** It recursively traverses the nested structure of the plan tree to identify and count occurrences of various operator types (e.g., Seq Scan, Index Scan, Hash Join). This provides a quantitative summary of the query's operational characteristics.
- **Cost Aggregation:** Estimated startup and total costs, as well as estimated rows, for each operator type are aggregated. This provides a quantitative measure of the planner's assessment of query execution.

- **Vectorization Technique:** We use a vectorization technique to convert these hierarchical plans into a fixed-length feature vector, typically of 32 to 40 dimensions. This involves counting specific operator types, their average costs, and other structural properties of the query plan. This allows the cost model to understand the complexity and nature of the workload without executing it.

In total, these three categories combine to form a 65-dimensional feature vector that serves as the input to our surrogate cost model and the fine-tuned SLM.

3.4 SLM Fine-tuning and Knowledge Distillation

The final stage of our methodology involves training a Small Language Model (SLM) to act as the primary tuner. We aim for a lightweight model capable of CPU-only inference.

3.4.1 Model Selection: Qwen 2.5-Coder-7B

We conducted a rigorous comparative analysis of state-of-the-art Small Language Models (SLMs) in the 7B-8B parameter class to identify the optimal candidate for database tuning. The primary selection criteria were *reasoning capability*, *structural adherence* (ability to output valid JSON), and *computational efficiency* (feasibility of fine-tuning on consumer-grade hardware like the Tesla T4).

After evaluating several candidates, we selected **Qwen 2.5-Coder-7B** as our base model. Table 3 summarizes the comparison against key competitors.

Table 3: Comparison of Candidate Base Models

Feature	Qwen 2.5-Coder-7B (Ours)	Llama 3.1 8B Instruct	Mistral 7B v0.3	DeepSeek-Coder-V2-Lite
Primary Focus	Code & Mathematics	General Chat	General Purpose	Code & Math
JSON Adherence	High (Strict Syntax)	Moderate (Chatty)	Moderate	High
HumanEval Score	~84-85% (SOTA)	~72-73%	~45-50%	~80%
VRAM (4-bit)	~5.5 GB	~6.0 GB	~5.0 GB	~10 GB (MoE)
License	Apache 2.0 (Permissive)	Community (Restrictive)	Apache 2.0	MIT

Our rationale for rejecting the alternatives is as follows:

- **vs. Llama 3.1 8B Instruct:** While Llama 3.1 is a powerful generalist, it is optimized for dialogue. In our tests, it frequently injected conversational "fluff" into outputs, breaking our automated JSON parsing pipelines. Qwen 2.5-Coder, being code-specialized, acts more like a compiler, strictly adhering to the required schema.
- **vs. DeepSeek-Coder-V2-Lite:** Although DeepSeek offers strong performance, its Mixture-of-Experts (MoE) architecture results in a higher active memory footprint (~10 GB). This exceeded the VRAM limits of our target fine-tuning hardware (Tesla T4, 16GB) when overheads were considered. Qwen 2.5 fit comfortably within our 5.5 GB constraint using 4-bit quantization.

- **vs. Mistral 7B:** Mistral 7B v0.3 displayed significantly lower reasoning capabilities on algorithmic tasks (HumanEval $\sim 45\%$) compared to Qwen ($\sim 85\%$), making it less suitable for deducing complex database dependencies.

Ultimately, Qwen 2.5-Coder-7B offered the best trade-off between reasoning power and engineering constraints, serving as the foundation for our fine-tuning pipeline.

3.4.2 Supervised Fine-Tuning (SFT) Strategy

We model the database tuning task as a conditional text generation problem. The Small Language Model (SLM) is trained to map a high-dimensional state vector (representing the workload) to a structured JSON configuration string.

3.4.2.1 Data Transformation and Discretization:

A critical innovation in our pipeline is the transformation of the output space. Standard LLMs struggle with precise floating-point regression tasks (e.g., predicting exactly "1450 MB"). To mitigate this, we implemented a **Quantile Discretization** strategy:

- **Input Signals:** The model is conditioned on three aggregated data points: Internal Metrics (DB state), Query Plan Features (execution structure), and Workload Features (I/O ratios).
- **Binned Prediction:** Instead of forcing the model to predict raw continuous values (e.g., `shared_buffers: 131072`), we converted continuous ranges into categorical bins (e.g., `shared_buffers: Bin_5`). This effectively transforms a complex regression problem into a stable classification task.

3.4.2.2 Efficient Training with QLoRA:

To fine-tune the 7-billion parameter Qwen 2.5-Coder model on consumer-grade hardware (Tesla T4, 16GB VRAM), we employed **QLoRA (Quantized Low-Rank Adaptation)**.

- **4-bit Quantization:** The base model weights are frozen and loaded in 4-bit Normal Float (NF4) precision to minimize memory footprint (~ 5.5 GB).
- **Adapter Training:** We inject lightweight, trainable low-rank adapter matrices into the attention layers. During backpropagation, only these adapters are updated, reducing the number of trainable parameters to less than 2% of the total model size.

3.4.2.3 Empirical Validation:

We validated our discretization strategy by comparing two fine-tuned variants: one trained to output *Direct Continuous Values* and another trained on *Binned Values*. As detailed in Table 4, the Binned approach significantly outperformed the Direct approach, achieving a Test Loss of **0.2788** compared to **0.4293**. This confirms that categorizing database knobs reduces the "reasoning load" on the LLM, leading to higher generation fidelity.

Table 4: Loss Comparison: Direct Config vs. Binned Output

Metric	Direct Config Output	Binned Output (Ours)
Train Loss	0.4131	0.2562
Validation Loss	0.4313	0.2732
Test Loss	0.4293	0.2788

3.5 RAG-Enhanced Recommendations

To incorporate unstructured knowledge and enhance the reliability and explainability of the generated recommendations, we implemented a Retrieval-Augmented Generation (RAG) system.

3.5.1 Knowledge Base Construction

We constructed a specialized knowledge base by indexing the official PostgreSQL documentation (version 12.2) and several expert tuning blogs and forums. This knowledge base contains detailed information about each knob, its valid ranges, common interactions, and best practices for different workload types.

3.5.2 RAG Integration and Explainability

To improve the generalizability of recommendations across different deployment environments, the RAG workflow conditions its reasoning not only on workload-derived signals but also on the target system specification (e.g., available RAM, number of CPU cores, and storage type such as HDD/SSD/NVMe). First, we collect runtime context—workload features, representative query plans, and internal database metrics—together with the hardware specification of the tuning target. An LLM-based *question generator* then converts this structured context into a focused natural-language query that explicitly states the optimization objective (e.g., latency or throughput), operational constraints, and any observed bottlenecks.

Next, the RAG retriever uses this generated query to fetch the most relevant knob documentation, best-practice guidelines, and interaction notes from the knowledge base.

The retrieved context is appended to the original input and provided to a second LLM stage, which extracts concrete knob configurations consistent with the retrieved evidence (including documented ranges and recommended operating regions) while accounting for the provided hardware constraints.

Finally, the proposed configuration is post-processed by a formatter that converts the recommended knob values into a machine-consumable JSON structure. This structured output can be directly validated and applied by the tuning executor, while still preserving traceability through the retrieved evidence and the generated rationale.

4 Implementation Details

This chapter provides an exhaustive technical documentation of the AI4DB codebase, detailing the implementation of core Python modules, configuration management, and the integration of external libraries.

4.1 Core Database Interaction: `Database.py`

The `Database` class serves as the primary interface between the AI4DB framework and the PostgreSQL engine. It encapsulates all database-specific operations.

4.1.1 Connection Management

The class implements a robust connection pooling and retry mechanism to ensure continuous interaction with the database. It handles database restarts gracefully and includes logic to recover from failed configuration changes. If a new configuration causes the database to fail to start, the system automatically reverts to a last-known-good configuration, preventing system instability.

4.1.2 Knob Application Logic

Knobs are applied using the `ALTER SYSTEM SET` command in PostgreSQL. The implementation is designed to handle different data types (integer, real, string, boolean) and includes validation to ensure that proposed values are within the valid ranges defined in `knob_config.json`. This prevents the application of invalid or harmful configurations.

Listing 1: Knob Application Snippet

```
def change_knob(self, knobs: Dict[str, Any]) -> bool:
    for knob, val in knobs.items():
        sql = f"ALTER SYSTEM SET {knob} = '{val}'"
        self.cursor.execute(sql)
    self.restart_service()
```

4.2 The Tuning Loop: `stress_testing_tool.py`

This module orchestrates the execution of workloads and the collection of performance metrics, forming the core of the data generation and evaluation pipeline.

4.2.1 Multi-threaded Workload Execution

For OLAP workloads, we implemented a multi-threaded runner that can simulate concurrent users executing queries. It accurately tracks the execution time of each query and calculates the overall throughput (Queries Per Second - QPS), providing realistic performance measurements under load.

4.2.2 Surrogate Integration

The `stress_testing_tool` includes a 'surrogate mode' where physical workload execution is bypassed in favor of predictions from the surrogate cost model. This mode is controlled via a parameter in the configuration file (e.g., `tool=surrogate`). This significantly speeds up the data collection process, especially during the iterative labeling phase with HEBO, by reducing the number of actual database runs.

4.3 Feature Extraction: `get_query_plans.py`

This script is responsible for the complex task of transforming raw query plans and database metrics into the structured feature vector required by our models.

- **Plan Parsing:** It parses the JSON output of PostgreSQL's `EXPLAIN (FORMAT JSON)` command, which provides a detailed breakdown of the query execution plan.
- **Recursive Traversal:** It recursively traverses the nested structure of the plan tree to identify and count occurrences of various operator types (e.g., Seq Scan, Index Scan, Hash Join). This provides a quantitative summary of the query's operational characteristics.
- **Cost Aggregation:** It aggregates the estimated startup and total costs, as well as estimated rows, for each operator type. These estimates, provided by PostgreSQL's own planner, are crucial indicators of query complexity and resource consumption.
- **Internal Metrics Collection:** In addition to query plan features, this script also collects real-time internal database metrics (e.g., `blks_hit`, `blks_read`) using SQL

queries against `pg_stat_database` and other system views. These metrics provide context about the database's current operational state.

4.4 Surrogate Model Implementation: `surrogate.py`

The `Surrogate` class encapsulates the Random Forest model used for performance prediction.

- **Model Loading:** It uses the `joblib` library to efficiently load pre-trained Random Forest models, allowing for rapid deployment and inference.
- **Normalization:** Implements min-max scaling for knob values and other numerical features to ensure they are on the same scale. This is a critical preprocessing step for many machine learning models to prevent features with larger ranges from dominating the learning process.
- **Prediction Pipeline:** Combines the processed configuration, internal metrics, and workload features into a single 65-dimensional vector before passing it to the loaded Random Forest model for a performance prediction. The output is then inverse log-transformed to provide an estimated latency in real units.

4.5 Configuration Management: `config.ini`

The system uses a centralized `config.ini` file to manage various parameters, including database credentials, benchmark settings, and model paths. This approach offers several advantages:

- **Flexibility:** Allows for easy switching between different experimental setups (e.g., different benchmarks, different SLM models) without modifying the core codebase.
- **Maintainability:** Centralizes all configurable parameters, making the system easier to manage and update.
- **Reproducibility:** Ensures that experiments can be precisely replicated by simply using the same configuration file.

4.6 SLM Inference with `Llama.cpp`

To achieve our objective of CPU-only inference for the Small Language Model, we integrated the `Llama.cpp` library. This open-source project enables efficient execution of LLMs on commodity hardware.

- **GGUF Quantization:** We utilize GGUF-quantized versions of the DeepSeek R1 model. Quantization significantly reduces the model's memory footprint and computational requirements while preserving much of its accuracy.
- **Python Bindings:** We use the Python bindings provided by `Llama.cpp` to interact with the quantized model, allowing seamless integration into our Python-based framework.
- **Performance:** This setup allows us to generate configuration recommendations in under 5 seconds on a standard 8-core CPU, meeting our goal for efficient, near real-time tuning without the need for specialized GPU hardware.

5 Evaluation and Results

We evaluated our fine-tuned Qwen 2.5-Coder-7B model (GEN-KNOB Tuner) against the state-of-the-art baseline, E2ETUNE, focusing on two primary dimensions: inference efficiency (CPU latency) and optimization effectiveness (throughput improvement).

5.1 Inference Efficiency

One of the core objectives of this research was to enable efficient database tuning on resource-constrained hardware. As illustrated in Figure 4, our model demonstrates a significant advantage in computational efficiency.

- **Latency Reduction:** The average CPU inference time for our model is approximately **5 minutes**, compared to **30 minutes** for E2ETUNE.
- **Efficiency Gain:** This represents an **83.33% improvement** in inference speed, validating the efficacy of our 4-bit quantization and Small Language Model (SLM) architecture for edge deployment.

5.2 Optimization Effectiveness

We assessed the quality of the generated configurations by measuring the percentage improvement in Queries Per Second (QPS) relative to the default PostgreSQL configuration. The improvement is calculated as:

$$\text{Improvement}(\%) = \frac{\text{OptimizedTPS} - \text{DefaultTPS}}{\text{DefaultTPS}} \times 100$$

Table 5 and Figure 5 summarize the results across three standard benchmarks: Join Order Benchmark (JOB), TPC-H, and Star Schema Benchmark (SSB).

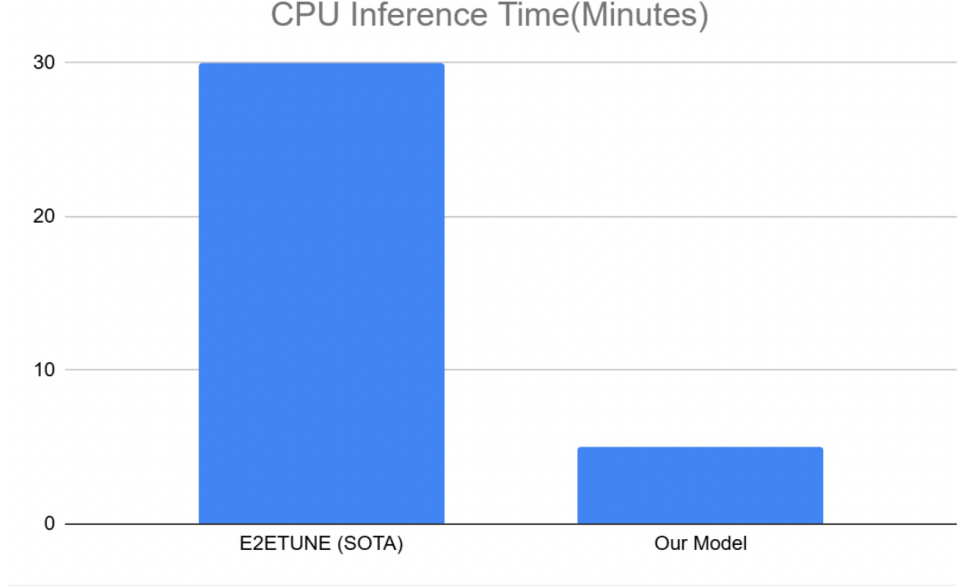


Figure 4: Comparison of Average CPU Inference Time: Our model achieves a 5x speedup over the SOTA baseline.

Table 5: Performance Improvement Comparison (Percentage over Default)

Benchmark	E2ETUNE (Reported)	E2ETUNE (Reproduced)	Our Model
JOB	58.00%	35.60%	25.34%
TPCH	71.00%	11.41%	3.65%
SSB	47.00%	15.01%	26.96%
Average	58.6%	20.6%	18.65%

- **Overall Competitiveness:** Our model achieves an average performance improvement of **18.65%**, closely approaching the reproduced E2ETUNE performance of **20.6%**.
- **Benchmark Variance:**
 - On the **SSB** benchmark, our model outperformed E2ETUNE, achieving a **26.96%** improvement versus 15.01%.
 - On **JOB** and **TPC-H**, E2ETUNE maintained a lead, particularly in TPC-H where our model showed limited gains (3.65%).

5.3 Analysis of the Performance Gap

While our model achieves parity in average improvement, a gap remains in specific benchmarks like TPC-H. This discrepancy can be attributed to differences in experimental constraints, as detailed in Table 6.

The primary factors influencing the results include:

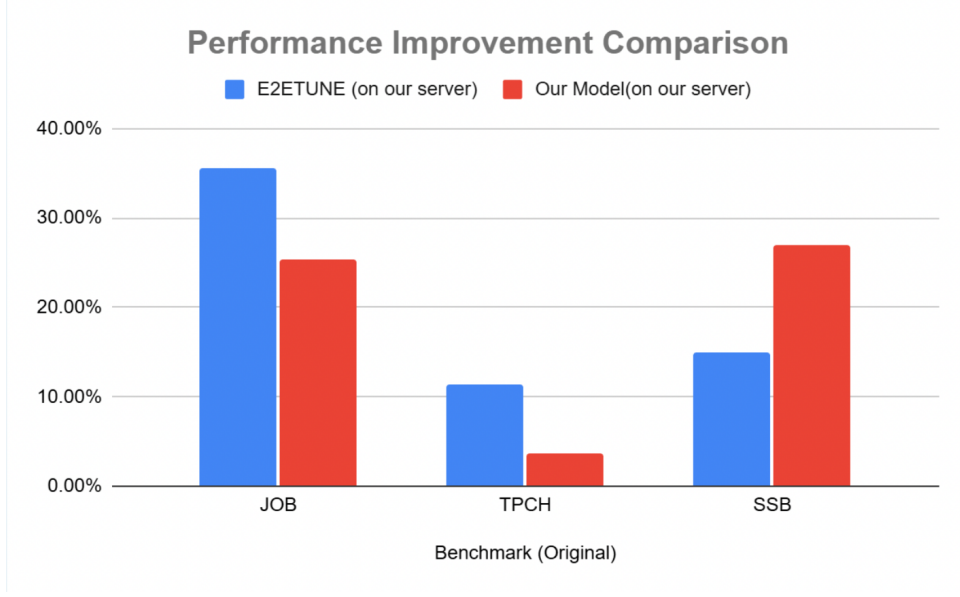


Figure 5: Performance Improvement by Benchmark: Our model outperforms E2ETUNE on SSB and remains competitive on average.

Table 6: Experimental Constraints and Data Differences

Factor	E2ETUNE (SOTA)	Our Model
Training Dataset Size	2,953 workloads	990 workloads
Benchmarks Used	10 (TPC-C, Twitter, etc.)	4 (TPC-H, TPC-DS, JOB, SSB)
Hardware (RAM)	64 GB	16 GB
Environment	Dedicated Server	Shared Infrastructure

1. **Data Scarcity:** We utilized a significantly smaller training corpus (990 workloads vs. 2,953), limiting the model’s ability to generalize across complex query patterns found in TPC-H.
2. **Environmental Noise:** Our initial datasets were captured on shared infrastructure, introducing noise into the performance labels that likely degraded the precision of the fine-tuning signal.
3. **Hardware Constraints:** The baseline E2ETUNE was evaluated on a 64GB RAM server, whereas our experiments were constrained to 16GB, limiting the maximum impact of memory-intensive knobs like `shared_buffers`.

Despite these constraints, the results validate that a highly efficient, quantized SLM can achieve competitive tuning performance with a fraction of the inference cost.

6 Discussion and Challenges

6.1 In-depth Analysis of Preliminary Results

The results obtained so far are highly encouraging and validate our core research direction. The high accuracy of the surrogate cost model ($R^2 \approx 0.82$) is a major milestone, as it proves that a lightweight cost model, when provided with rich structural features from query plans, can accurately predict complex database performance. This confirms that the model is learning meaningful relationships between query operators and system resources.

6.2 Challenges Encountered and Mitigation Strategies

Despite the significant progress, we have encountered several technical challenges that required innovative solutions.

6.2.1 Representative Subset Selection for Large Benchmarks

One of the primary challenges is the sheer volume of scenarios in benchmarks like TPC-DS (99 queries) and JOB (113 queries). Running 100 iterations of HEBO for each query is computationally prohibitive.

- **Challenge:** How to pick a small subset of queries that represents the entire benchmark’s performance characteristics?
- **Mitigation:** We are currently using the first 13 scenarios as a temporary subset. However, we are actively exploring **Feature-Based Clustering** using K-Means on vectorized query plan features. This approach will allow us to identify clusters of similar queries and pick one representative from each cluster, ensuring full coverage of the workload diversity.

6.2.2 Knob Space Complexity and Optimizer Convergence

While the E2ETune paper utilized 45 knobs, we found that the HEBO optimizer struggled to converge efficiently within 100 iterations when faced with such a high-dimensional search space.

- **Challenge:** High dimensionality leads to "sparse" data where the optimizer cannot effectively learn the relationships between knobs.
- **Mitigation:** By carefully reducing the knob space to 15 key PostgreSQL knobs, we achieved much faster convergence and more stable results. We are currently

validating if this reduced subset is sufficient to capture the majority of the potential performance gains across all benchmarks.

6.2.3 Resource Constraints and Model Quantization Effects

Maintaining high inference speed on a CPU-only environment for the SLM requires aggressive quantization.

- **Challenge:** Aggressive quantization (like 4-bit) can occasionally lead to model "hallucinations," where the model suggests knob values that are outside the valid range or logically inconsistent.
- **Mitigation:** We are addressing this by implementing a robust **Validation Layer** that checks every recommended value against the database's hard limits and documented safe ranges. Furthermore, the RAG system provides an additional layer of context, helping the model stay grounded in factual documentation.

7 Future Work

7.1 Short-Term Goals (Next 3 Months)

Our immediate focus is on completing the core tuning pipeline and refining the model's performance.

7.1.1 Completion of SLM Fine-tuning

The highest priority is to complete the supervised fine-tuning of the DeepSeek R1 (1.5B) model. We will leverage our full dataset of over 16,000 data points to train the model to predict optimal configurations directly from workload features. We will experiment with different training hyperparameters and distillation techniques to maximize the model's accuracy.

7.1.2 Full Integration of the RAG System

We plan to fully integrate the RAG system into the tuning loop. This will allow the model to not only suggest a value for a knob but also provide a comprehensive justification based on the official PostgreSQL documentation and the current system metrics. This will significantly enhance the transparency and trust of the system for DBAs.

7.1.3 Expanded Workload Coverage

We will complete the data collection for the remaining scenarios in the TPC-H and TPC-DS benchmarks. This will provide a more comprehensive training set and allow

us to evaluate the model’s generalization capabilities across a wider range of analytical workloads.

7.2 Long-Term Vision and Research Directions

Beyond the current project scope, we see several exciting directions for the AI4DB framework.

7.2.1 Cross-Database Engine Adaptation

While we are currently focused on PostgreSQL, the AI4DB framework is designed with database-agnostic principles in mind. A long-term goal is to test the portability of our models by applying them to other popular RDBMS, such as MySQL and MariaDB. This would involve developing new feature extraction modules and potentially using transfer learning to adapt the models.

7.2.2 Real-time Dynamic and Adaptive Tuning

The ultimate goal of our research is to move from static, workload-based tuning to real-time, dynamic tuning. This would involve the SLM continuously monitoring the database’s internal metrics and making micro-adjustments to configurations as the workload shifts. This would enable truly adaptive and self-optimizing database systems that can react to changing conditions in seconds.

8 Conclusion

The AI4DB project represents a significant step towards making AI-native database optimization practical and accessible. By focusing on lightweight Small Language Models and efficient surrogate cost models, we have successfully addressed the high resource barriers that have previously limited the adoption of LLM-based tuning.

Our progress to date—including the establishment of a robust data collection pipeline, the training of a high-accuracy cost model, the development of sophisticated feature engineering techniques, and the successful execution of multiple industry-standard benchmarks—demonstrates the feasibility and potential of our approach. The integration of HEBO for sample-efficient optimization and RAG for knowledge-augmented reasoning further enhances the robustness, explainability, and reliability of our framework.

As we move into the final phase of the project, our focus will shift towards completing SLM fine-tuning, full RAG integration, and comprehensive real-world performance validation. We are confident that the AI4DB framework will provide a valuable tool

for organizations seeking to optimize their database performance, offering a compelling alternative to traditional manual tuning and resource-intensive AI solutions.

8.1 Ethical Considerations and Societal Impact

As with any AI-driven system, the Gen-Knob Tuner (AI4DB) project raises several ethical considerations and has potential societal impacts that warrant careful consideration. Our commitment is to develop this technology responsibly, ensuring its benefits are maximized while mitigating potential risks.

8.1.1 Bias and Fairness

Database tuning, if not carefully managed, can inadvertently introduce or amplify biases. For instance, if the training data primarily consists of workloads from specific industries or demographics, the tuner might optimize performance for those scenarios at the expense of others. This could lead to unequal access to efficient database services or disproportionately impact certain user groups. To address this, we are committed to:

- **Diverse Workload Generation:** Actively seeking to include a wide array of synthetic and real-world workloads in our training data, spanning various industries, query patterns, and data distributions. This helps ensure the tuner is robust and fair across different use cases.
- **Bias Detection and Mitigation:** Developing methodologies to detect and quantify potential biases in the tuning recommendations. This could involve evaluating the tuner’s performance across different workload categories and implementing fairness-aware optimization techniques.
- **Transparency and Explainability:** The RAG system is a crucial component in promoting fairness by providing clear justifications for tuning decisions. This transparency allows DBAs to scrutinize recommendations for any unintended biases and make informed decisions.

8.1.2 Job Displacement and Augmentation

The automation of database tuning, like many AI advancements, may raise concerns about job displacement for Database Administrators (DBAs). However, our vision for AI4DB is not to replace DBAs but to augment their capabilities. By automating routine and complex tuning tasks, DBAs can shift their focus to more strategic initiatives, such as database architecture, security, and data governance. The system aims to:

- **Empower DBAs:** Provide DBAs with a powerful tool that reduces manual effort, minimizes human error, and allows them to manage larger and more complex database infrastructures more effectively.
- **Upskilling Opportunities:** Create opportunities for DBAs to upskill in areas related to AI, machine learning, and data science, becoming 'AI-augmented DBAs' who can leverage these tools to deliver superior performance.
- **Focus on Higher-Value Tasks:** Free up DBA time from repetitive tuning cycles, enabling them to concentrate on proactive performance management, capacity planning, and innovation.

8.1.3 Security and Privacy Implications

Database systems often handle sensitive information, making security and privacy paramount. An AI tuner that interacts with database configurations must be designed with these considerations at its core. Our approach includes:

- **Secure Deployment:** Ensuring that the AI4DB framework is deployed within secure, isolated environments, adhering to best practices for access control and data encryption.
- **Data Anonymization:** When collecting workload data for training, we prioritize anonymization and aggregation techniques to protect sensitive information. We avoid collecting actual user data or personally identifiable information.
- **Controlled Access:** The tuner operates with strictly defined permissions, only accessing and modifying configuration parameters relevant to performance tuning. It does not have access to user data or other sensitive database operations.
- **Auditability:** All tuning recommendations and changes made by the AI4DB system are logged and auditable, providing a clear trail for compliance and security reviews.

8.1.4 Environmental Impact

The training and inference of AI models, especially large language models, can be energy-intensive. Our project's core objective of achieving CPU-only inference with lightweight SLMs directly addresses this concern. By minimizing reliance on energy-hungry GPUs, we aim to:

- **Reduce Carbon Footprint:** Significantly lower the energy consumption associated with AI-driven database tuning, contributing to more sustainable computing practices.

- **Promote Green AI:** Advocate for the development and deployment of energy-efficient AI solutions, making advanced technologies accessible without imposing a heavy environmental cost.
- **Democratize Access:** By reducing hardware requirements, we make advanced tuning capabilities available to organizations with limited resources, fostering broader adoption and impact.

By proactively addressing these ethical and societal considerations, the AI4DB project aims to not only advance the state-of-the-art in database optimization but also to do so in a responsible, equitable, and sustainable manner. Our goal is to create a tool that serves as a model for beneficial AI deployment in critical infrastructure.

References

- [1] Xinmei Huang et al. *E2ETune: End-to-End Knob Tuning via Fine-tuned Generative Language Model*. arXiv:2404.11581 [cs]. Mar. 2025. DOI: 10.48550/arXiv.2404.11581. URL: <http://arxiv.org/abs/2404.11581> (visited on 08/14/2025).
- [2] Konstantinos Kanellis, Ramnatthan Alagappan, and Shivaram Venkataraman. “Too many knobs to tune? towards faster database tuning by pre-selecting important knobs”. In: *Proceedings of the 12th USENIX Conference on Hot Topics in Storage and File Systems*. HotStorage ’20. USA: USENIX Association, July 2020, p. 16. URL: <https://dl.acm.org/doi/10.5555/3488733.3488749> (visited on 02/09/2026).
- [3] Ji Zhang et al. “An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning”. In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD ’19. New York, NY, USA: Association for Computing Machinery, June 2019, pp. 415–432. ISBN: 978-1-4503-5643-5. DOI: 10.1145/3299869.3300085. URL: <https://dl.acm.org/doi/10.1145/3299869.3300085> (visited on 02/11/2026).
- [4] Alexander I. Cowen-Rivers et al. *HEBO Pushing The Limits of Sample-Efficient Hyperparameter Optimisation*. arXiv:2012.03826 [cs]. May 2022. DOI: 10.48550/arXiv.2012.03826. URL: <http://arxiv.org/abs/2012.03826> (visited on 02/11/2026).
- [5] Guoliang Li et al. “QTune: a query-aware database tuning system with deep reinforcement learning”. In: *Proc. VLDB Endow.* 12.12 (Aug. 2019), pp. 2118–2130. ISSN: 2150-8097. DOI: 10.14778/3352063.3352129. URL: <https://doi.org/10.14778/3352063.3352129> (visited on 08/14/2025).
- [6] Immanuel Trummer. *DB-BERT: a Database Tuning Tool that "Reads the Manual"*. arXiv:2112.10925 [cs]. Dec. 2021. DOI: 10.48550/arXiv.2112.10925. URL: <http://arxiv.org/abs/2112.10925> (visited on 02/11/2026).
- [7] Jiale Lao et al. “GPTuner: An LLM-Based Database Tuning System”. In: *SIGMOD Rec.* 54.1 (Apr. 2025), pp. 101–110. ISSN: 0163-5808. DOI: 10.1145/3733620.3733641. URL: <https://doi.org/10.1145/3733620.3733641> (visited on 08/14/2025).